Skill Issue – A Peer-to-Peer Skill Exchange Platform

CSLR31 Data Structures Laboratory – Project Report

Team: All H's

106125037 – Harini
106125039 – Harsh
106125041 – Himani

November 3, 2026

Contents

1 Step 1 – Problem Identification (Abstract) 2

2 Step 2 – Design of the Product 2
2.1 Idea . . . 2
2.2 Architecture and modules . . . 2
2.3 Working of the product . . . 4
2.4 Data structures and complexity . . . 5

3 Step 3 – Coding 5
3.1 Directory structure . . . 5
3.2 Sample code . . . 5
3.3 Known limitations . . . 6

4 Step 4 – Testing 6

5 Step 5 – Contributions, Key Takeaways, Future Scope, Acknowledgements and References 7
5.1 Individual contributions . . . 7
5.2 Key takeaways . . . 8
5.3 Future scope . . . 8
5.4 Acknowledgements . . . 8
5.5 References . . . 9

1 Step 1 – Problem Identification (Abstract)

Title: Skill Issue – A Peer-to-Peer Skill Exchange Platform.
Problem and motivation. Students on a campus hold very different skills (programming, music, dance, languages), yet there is no simple way to find a peer who can teach a skill, agree on a time, and be recognised for teaching. Paid courses are expensive and generic, so many students stay stuck on a skill they could learn from a classmate. This supports UNESCO/UN **SDG 4 (Quality Education)** by making peer learning accessible and free. **Existing products** such as Coursera, Udemy and Superprof are paid or one-directional marketplaces; campus WhatsApp groups are unstructured and keep no record of sessions or reputation. **Objectives:** let users register their teachable skills, find mentors for a skill, request and schedule sessions, track attendance, and reward good teaching. **Salient features:** (i) skill-based mentor matching ranked with a hand-written stable merge sort; (ii) a booking life-cycle (PENDING → OFFERED → CONFIRMED → session) with mentor time offers; (iii) attendance reporting with no-show disputes resolved by an admin; (iv) points system (learner ratings and quiz scores) feeding a teaching leaderboard; (v) self-written hash map for $O(1)$ look-ups and a self-written heap for top-k mentors; (vi) CSV persistence, a console interface and a REST API consumed by a React web front end. **Future scope:** a skill graph with trie-based skill search, interval-based time-slot conflict detection with a request queue, password hashing, notifications, a database back end and mobile support.

2 Step 2 – Design of the Product

Figure 1 shows the flow chart of the main workflow. Grey boxes mark the steps where a data structure or algorithm does the real work. (If `figures/flowchart.pdf`, exported from `flowchart.dot`/`.fig`, is present it is used; otherwise the built-in diagram below is drawn, so the report always compiles.)

2.1 Idea

Every user is both a potential *learner* and a potential *mentor*. A learner searches for a skill; the system lists matching mentors ranked by reputation (teaching points). The learner sends a booking request to one mentor, the mentor replies with a time, and the learner confirms. After the session the learner rates the mentor, the mentor grades a short quiz of the learner, and both feed the points that drive the leaderboard.

2.2 Architecture and modules

The product is split into seven modules; each has a header (declaration) and a source file (definition).

Models (`models.h`)
Plain records `User`, `BookingRequest`, `MentorOffer`, `BookingDetails` and `Session`. They carry only data, so every other module can include them without circular dependencies.

Storage (`storage.h/.cpp`)
Class `Storage` loads and saves the four CSV files (`users`, `booking_requests`, `mentor_offers`, `sessions`). Fields are quoted and escaped, and stray carriage returns (Windows line endings) are ignored when reading; malformed rows are skipped; a missing file is treated as empty data.

Sorting (`sorting.h`)
A templated, stable merge sort taking a comparator. It is used for the leaderboard and mentor ranking.

Hash map (`hashmap.h`)
A self-written hash map (separate chaining, djb2 string hash, doubling when the load factor

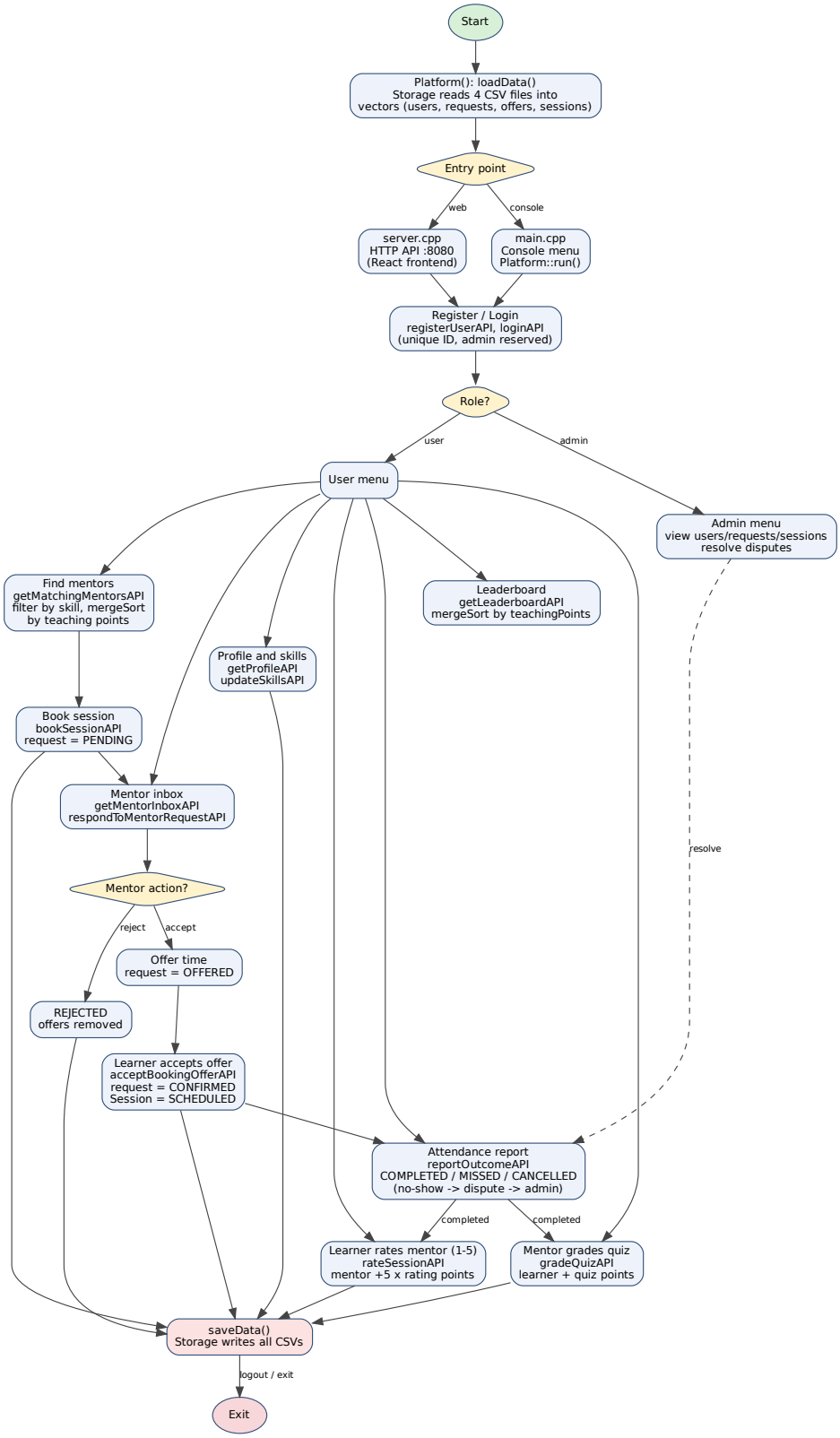


Figure 1: Flow chart of the Skill Issue platform.

exceeds 0.75). `Platform` uses it as an index from user ID and request ID to the position in the corresponding vector, giving $O(1)$ average look-ups.

Priority queue (`priority_queue.h`)
A self-written binary heap stored in a `std::vector`, using the same comparator convention as the merge sort. It powers the top-k mentors feature.

Platform (`platform.h/.cpp`)
The core logic. On construction it loads all CSVs into four `std::vector`s. It also owns the hash indexes `userIndex_` and `requestIndex_`, which are rebuilt after loading and updated on every registration and booking. Public `*API` functions (register, login, book, respond, accept, report outcome, rate, grade quiz, leaderboard...) return a success flag and a message and are shared by both interfaces. Every state change calls `saveData()`.

Interfaces
`main.cpp` gives a text menu (`Platform::run`); `server.cpp` wraps the same `Platform` in REST endpoints on port 8080 using the third-party header `cpp-httplib`; the React app in `frontend/` calls these endpoints.

2.3 Working of the product

1. Start-up and authentication. `Platform()` reads the CSV files and computes the next free request and session IDs. `registerUserAPI` rejects empty IDs, the reserved ID `admin`, and duplicate IDs; `loginAPI` compares the ID and password.

2. Mentor discovery. `getMatchingMentorsAPI(skill, learner)` scans all users, keeps those whose semicolon-separated skill list contains the skill (case-insensitive) and who are not the learner, and sorts them with `mergeSort`: higher teaching points first, then more teaching sessions, then smaller ID.

3. Booking life-cycle. A request moves through a small state machine:

$$\texttt{PENDING} \xrightarrow{\text{mentor offers a time}} \texttt{OFFERED} \xrightarrow{\text{learner accepts}} \texttt{CONFIRMED}$$
$$\texttt{PENDING} \text{ or } \texttt{OFFERED} \xrightarrow{\text{mentor rejects}} \texttt{REJECTED}$$

`bookSessionAPI` validates mentor, learner and skill and creates the request. `respondToMentorRequestAPI` lets *only the targeted mentor* reject (which also deletes its offers) or offer a time. `acceptBookingOfferAPI` turns the offer into a confirmed time, creates a `SCHEDULED Session` and increments the teaching/learning counters.

4. Attendance and disputes. After the scheduled time the participants report the outcome. In the console flow, a no-show report puts the session in `AWAITING_CONFIRMATION`; the other side can agree (5 points are deducted from the absentee) or disagree, which sets `DISPUTED` for the admin to resolve (completed, learner absent, mentor absent, or reschedule).

Top mentors (heap). `getTopMentorsAPI(k)` keeps a heap of at most k users whose top element is the *worst* user kept so far. Each user is pushed and, if the heap exceeds k, the worst is popped, so the cost is $O(n \log k)$ instead of sorting all n users. The result is reversed to give best-first order and is exposed as `GET /api/top-mentors?k=3`.

5. Points and leaderboard. A learner rating $r \in [1, 5]$ gives the mentor $5r$ teaching points (console flow); the mentor's quiz mark adds to the learner's total points. `getLeaderboardAPI` sorts all users by teaching points (ties by ID) and strips passwords before returning them.

2.4 Data structures and complexity

Structure / algorithm	Used for	Cost
`std::vector` of records	In-memory tables for users, requests, offers, sessions	append $O(1)$ amortised
Hash map (own implementation)	`findUser`, `findRequest` through ID indexes	$O(1)$ average, $O(n)$ rebuild on load
Binary heap / priority queue (own)	Top-k mentors	push/pop $O(\log k)$, top-k in $O(n \log k)$
Merge sort (own implementation)	Leaderboard, mentor ranking	$O(n \log n)$, $O(n)$ extra space, stable
Erase–remove idiom	Deleting offers of a rejected request	$O(n)$
String tokenising (`;`)	Skill matching	$O(L)$ per user
State machine on strings	Request and session status	$O(1)$ per transition
CSV (flat file)	Persistence	$O(n)$ per save

Users and requests are only ever appended, never deleted, so the hash indexes only need insertions and a rebuild after loading; positions never shift. Linear scans remain for offers and sessions, which are filtered by several fields rather than looked up by one key.

3 Step 3 – Coding

3.1 Directory structure

```
All_Hs_106125037_106125039_106125041_CSLR31_Project/
|-- Makefile a.out (console) server (HTTP API)
|-- include/ models.h storage.h platform.h sorting.h hashmap.h
| priority_queue.h httplib.h
|-- Source/ main.cpp platform.cpp storage.cpp server.cpp
|-- object/ *.o files
|-- data/ users.csv booking_requests.csv mentor_offers.csv sessions.csv
|-- Test/ test_skill_issue.cpp testdata/
|-- frontend/ React + Vite web UI
|-- Document/ report.tex, user_manual.md, figures/
`-- Video/ project experience video
```

Build with `make`; run `./a.out` (console) or `./server` then `npm run dev` in `frontend/`. Declarations live in the `.h` files and definitions in the `.cpp` files, as required.

3.2 Sample code

Generic stable merge sort (`include/sorting.h`):

```
template <typename T, typename Compare>
void mergeSortHelper(std::vector<T>& arr, int left, int right, Compare comp) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSortHelper(arr, left, mid, comp);
    mergeSortHelper(arr, mid + 1, right, comp);
    std::vector<T> temp;
    int i = left, j = mid + 1;
    while (i <= mid && j <= right) {
        if (!comp(arr[j], arr[i])) temp.push_back(arr[i++]); // keeps ties stable
        else temp.push_back(arr[j++]);
    }
    while (i <= mid) temp.push_back(arr[i++]);
```

```
    while (j <= right) temp.push_back(arr[j++]);
    for (int k = 0; k < (int)temp.size(); k++) arr[left + k] = temp[k];
}
```

Use in `getMatchingMentorsAPI` (`Source/platform.cpp`):

```
mergeSort(mentors, [](const User& a, const User& b) {
    if (a.teachingPoints != b.teachingPoints) return a.teachingPoints > b.teachingPoints;
    if (a.teachingSessions != b.teachingSessions) return a.teachingSessions > b.teachingSessions;
    return a.id < b.id;
});
```

Core of the self-written hash map (`include/hashmap.h`):

```
void put(const K& key, const V& value) {
    Node* node = findNode(key);
    if (node) { node->value = value; return; } // overwrite existing key
    if (size_ + 1 > bucketCount_ * 3 / 4) resize(bucketCount_ * 2); // keep chains short
    std::size_t b = hashKey(key) % bucketCount_;
    buckets_[b] = new Node{key, value, buckets_[b]}; // insert at chain head
    ++size_;
}
```

Core of the self-written priority queue (`include/priority_queue.h`):

```
void push(const T& value) { data_.push_back(value); siftUp(data_.size() - 1); }
void pop() {
    data_.front() = data_.back(); // move last leaf to the root
    data_.pop_back();
    if (!data_.empty()) siftDown(0);
}
```

The project compiles with `-std=c++17 -Wall -Wextra -pedantic` without warnings.

3.3 Known limitations

- The API versions `rateSessionAPI` and `gradeQuizAPI` (used by the web app) store the rating/score but do not add the points that the console versions add; the quiz range is 0–100 in the API but 0–5 in the console.
- The API reports only the mentor's outcome; the dispute flow exists only in the console.
- Passwords are stored in plain text; the server trusts the user ID sent by the client (no session tokens).

4 Step 4 – Testing

Tests are in `Test/test_skill_issue.cpp`; run `make test`. They use their own data (`Test/testdata`) copied to a temporary directory, so real data is never modified. **Result: 73 passed, 0 failed**.

Test data. Five users: u1 (C++, Python; 40 pts, 3 sessions), u2 (Python, Web; 40 pts, 5 sessions), u3 (Python; 10 pts), u4 (Dance; 0 pts), u5 (Java; 25 pts).

IDs	Scenario	Expected
TC01–05	Register valid / duplicate / reserved `admin` / empty ID / empty password	Only the valid one succeeds
TC06–12	Login right/wrong/unknown; profile; update skills	Correct accept/reject; password hidden
TC13–16	Mentor search for "python" by u5, by u1, unknown learner, untaught skill	Order u2,u1,u3 (tie on points broken by sessions); self excluded; empty lists
TC17–18	Leaderboard	u1,u2,u5…; passwords blank
TC19–22	Booking: unknown mentor, self, skill not taught, valid	First three fail, last succeeds
TC23–30	Inbox and mentor response: target vs other mentor, bad action, no time, duplicate offer	Only target mentor can offer once
TC31–36	Learner accepts offer: wrong learner, wrong mentor, twice, counters	Session created once; counters +1
TC37–46	Outcome and rating: rating too early, bad outcome, wrong user, 0/6, double rating	Rejected except valid actions
TC47–50	Quiz grading: 101, non-mentor, valid, unknown session	Only valid succeeds
TC51–53	Reject flow	Request leaves inbox, cannot be accepted
TC54–55	Restart persistence	New user and sessions reloaded from CSV
TC56–59	Merge sort: empty, single, ascending, stable, 2000 random vs `std::stable_sort`	Identical results
TC60–64	Hash map: 1000 inserts with repeated resizing, overwrite, missing key, remove, integer keys	All keys found; size correct
TC65–67	Priority queue: empty queue, `top()` on empty (exception), push 7 values	Popped in priority order 9,9,7,5,3,2,1
TC68–71	Top-k mentors: $k = 2$, k larger than users, $k \leq 0$, agreement with leaderboard	Correct best-first order; empty for $k \leq 0$
TC72–73	CSV files with Windows (CRLF) line endings and a stray carriage return inside quoted values	Loaded cleanly; no hidden CR in IDs or times

Manual testing. The HTTP endpoints were additionally checked by hand with `curl` against the running server: registration (valid, duplicate, reserved, empty password), login, mentor search with a points tie, the full booking path (request, offer, acceptance, repeated acceptance), rating before and after completion, out-of-range ratings, quiz grading by a non-mentor, profile of an unknown user, and persistence of the rating and quiz values in `sessions.csv`. All 31 checks behaved as specified; the one finding was that a non-numeric numeric parameter (for example `requestId=abc`) raises an uncaught `std::stoi` exception and returns HTTP 500 instead of 400.

Not automatically tested. The interactive console menus, dispute resolution and the React UI.

5 Step 5 – Contributions, Key Takeaways, Future Scope, Acknowledgements and References

5.1 Individual contributions

The work was divided by area at the start of the project. The table states what each member's area produced in the delivered code; ideas planned for an area but not completed are listed under Future Scope and are not claimed here.

Member	Assigned area	Contribution in the delivered code
Himani 106125041	Skill discovery and matching	Skill-list handling and case-insensitive skill matching; matching-mentor search and its three-level ranking; update-teaching-skills feature; mentor-selection step of the booking flow; *Book a Session* and *Update Skills* pages of the web UI.
Harsh 106125039	Points, ranking and priority structures	Generic merge sort (`sorting.h`) and comparators; teaching leaderboard; own hash map (`hashmap.h`) and binary-heap priority queue (`priority_queue.h`); top-k mentors; teaching points from ratings, learner points from quiz marks, no-show penalties.
Harini 106125037	Booking life-cycle, sessions and persistence	Booking request, offer and confirmation flow; session records and states; attendance reporting, no-show confirmation, dispute and admin resolution; CSV storage layer (`storage.cpp`); REST server (`server.cpp`); project integration, Makefile and testing.

All three members took part in design discussions, testing and final integration of the console application, REST API and web UI.

5.2 Key takeaways

- Separating declarations (`.h`) from definitions (`.cpp`) let three people work on modules in parallel and kept the build incremental through a Makefile.
- A templated merge sort with a comparator gave one reusable, stable $O(n \log n)$ ranking routine for three different orderings.
- Modelling booking and session status as an explicit state machine made invalid actions (double accept, double rating) easy to detect and test.
- Implementing our own hash map and binary heap showed how chaining, load-factor resizing and sift-up/sift-down give $O(1)$ look-ups and $O(\log n)$ priority operations.
- Keeping one `Platform` class behind both a console UI and a REST API avoids duplicated rules, but it also showed what happens when the two paths drift apart (the point-awarding mismatch); automated tests exposed edge cases (ties, empty inputs, persistence) that manual clicking missed.

5.3 Future scope

- **Planned but not implemented:** a skill graph (user↔skill and skill↔skill adjacency lists) with BFS/DFS for "who can teach X" and compatibility scoring; a trie for skill-name autocomplete; interval scheduling for slot-conflict detection and timetable overlap; a queue that serves booking requests in priority order using the heap.
- Index offers and sessions by key as well (currently only users and requests use the hash map), and let a heap order the mentor inbox by urgency.
- Hash passwords and add token-based sessions.
- Detect overlapping appointments; add notifications; use a real database; port the dispute flow to the API; mobile app.

5.4 Acknowledgements

We thank our course instructor for the project framework and guidance, the Department of Computer Science and Engineering, NIT Tiruchirappalli, for the laboratory facilities, and the authors of the open-source tools we used: cpp-httplib, React, Vite, Graphviz and LaTeX (Overleaf).

5.5 References

1. T. H. Cormen et al., *Introduction to Algorithms*, 4th ed., MIT Press, 2022 (merge sort).
2. cpp-httplib, a C++ single-header HTTP library, https://github.com/yhirose/cpp-httplib (third-party code in `include/httplib.h`, not written by the team).
3. cppreference.com, C++17 standard library documentation, https://en.cppreference.com.
4. UNESCO/UN Sustainable Development Goals, https://www.unesco.org/en/sdgs.
5. React and Vite documentation, https://react.dev, https://vite.dev.
6. Graphviz, https://graphviz.org (flow chart generation).